Design and Analysis of Algorithms 2016
National University of Singapore CS3230
Prof Lee Wee Sun

Homework 3 (Due Date: Friday 28 October)

Please write the following on your homework:

- Name
- Collaborators (write none if no collaborators)
- Source, if you obtained the solution through research, e.g. through the web.

While you may collaborate, you *must write up the solution yourself* and you are asked to try doing the questions yourself first before discussing with others.

If you are asked to describe an algorithm, you should first provide a short description of the problem and what your main results are. Then you should provide a description of the algorithm in English, and if helpful, in pseudo-code. An example, or diagram can often be helpful in helping describe the algorithm. You should provide a proof (or indication) of correctness of the algorithm, and an analysis of the running time of the algorithm. Remember that you need to communicate clearly. Grades cannot be awarded if the grader is unable to understand what you are writing.

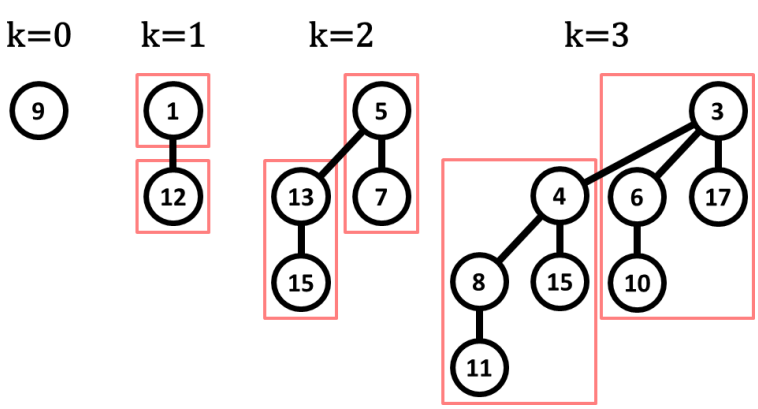
---

1. **Binomial Heap Insertion**
   A priority queue implemented using a min-heap has DELETE-MIN and INSERT operations that take $\Theta(\log n)$ time respectively. In this question, we will look at Binomial Heap, which can implement a priority queue with $\Theta(\log n)$ DELETE-MIN and amortised $\Theta(1)$ INSERT operations. A sequence of n insert operations take $\Theta(n \log n)$ time using a min-heap, but only $\Theta(n)$ time using a Binomial Heap.

   A Binomial Heap is implemented as a collection of Binomial Trees.
   A Binomial Tree is defined recursively as follows:

   - A binomial tree of order 0 is a single node.
   - A binomial tree of order k is formed from merging two binomial trees of order $k-1$.

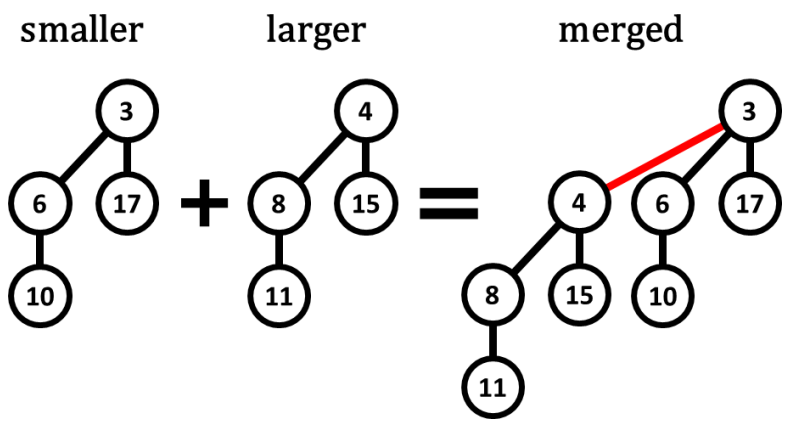
   Thus a binomial tree of order k has 2^k elements. The binomial trees for orders 0 to 3 are shown below.



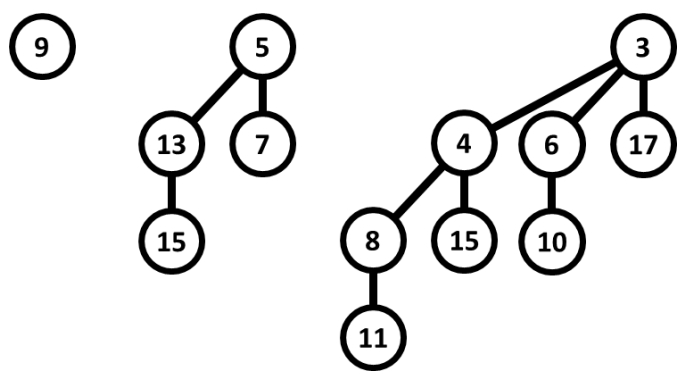
Similar to Binary Heaps, Binomial Trees have the additional property that each element in the tree is smaller than each of its children. We note that two Binomial Trees of the same size can be merged in $O(1)$.

To merge two Binomial Trees of the same size, we compare the roots of the two trees. The tree with the larger root is made a child of the root of the other tree.

An illustration is as follows:



Finally, a Binomial Heap (not a Binomial Tree!) is made out of a collection of Binomial Trees, all having different sizes.
For example, here is a Binomial Heap with 13 elements.
It consists of the trees of orders 0, 2 and 3.



(a) For a Binomial Heap containing 27 elements, what are the orders of the Binomial Trees that make up the Binomial Heap?

Solution: [1 Mark]
It is made up of trees of orders $0, 1, 3, 4$.

(b) For a Binomial Heap with n elements, give (and prove) an upper bound for the number of trees in the Binomial Heap. Also show that this upper bound is tight (i.e. it can be attained). Use exact rather than asymptotic notation for this question.

Solution: [3 Marks]

Upper bound: $\lfloor \log_2 n \rfloor + 1$
Proof: Cannot have a tree with size larger than n. Thus the largest possible order is largest k where $2^k \leq n$.
$\iff k \leq \log_2 n$
Thus the largest possible order is $\lfloor \log_2 n \rfloor$.
There are $k+1$ different trees of order $\leq k$.
Thus there can be at most $\lfloor \log_2 n \rfloor + 1$ trees.

Tight:
(Preferred)
Take any $k \in \mathbb{N}$. Consider $n = 2^k - 1$.
The orders of the trees are $0, 1, 2, \cdots, k-1$, which is k trees. $\lfloor \log_2 n \rfloor + 1 = k - 1 + 1 = k$.
(Partial mark for specific case - 0.5)
Let $n = 7$. Orders of trees are $0, 1, 2$. $\lfloor \log_2 n \rfloor + 1 = 3$.

Some Other Possible Solutions:
$\lceil \log_2(n+1) \rceil, \lfloor \log_2(n+1) \rfloor, \log_2(n+1)$
(all three tight at $2^k - 1$)

$\max\{\lceil \log_2(n-1) \rceil, 2\}$
(Correct, yet rather cringeworthy. Tight at $2^k - 1$)

This also shows that the minimum value of the Binomial Heap can be found in $O(\log n)$. (As the minimum-valued node is a root of one of the Binomial Trees)

INSERT
Inserting a new element v into the Binomial Heap is done as follows:

- First create a new tree of order 0 (containing a single node v)
- While there exists two trees of the same order, merge them into the tree of the next larger order.

This ensures that there is only one tree of each order after each insertion.
For each $k \in \{0, 1, 2, \cdots\}$, we define T_k to be the number of trees of order k. Each T_k is thus always either 0 or 1.

(c) Give the values of $(T_0, T_1, T_2, T_3, T_4)$ for a Binomial Heap with 27 elements.

Solution: [1 Mark] $(T_0, T_1, T_2, T_3, T_4) = (1, 1, 0, 1, 1)$

(d) Give the new values of $(T_0, T_1, T_2, T_3, T_4)$ for the Binomial Heap in (c) after adding one new element.

Solution: [1 Mark] $(T_0, T_1, T_2, T_3, T_4) = (0, 0, 1, 1, 1)$

(e) Use the Accounting or Potential method to prove that the INSERT operation runs in amortised $O(1)$.

Recommended Strategy:

- First define an invariant for how much "money" or "potential" is stored in the data structure based on the values of T_k.
- Then prove that the invariant holds after every operation, using a constant cost for each operation.

Solution: [4 Marks]
Potential Method:
Insert actual costs: 1 to add new node, 1 per merge.
A merge occurs whenever a T_k is set from 1 to 0.

Potential Function $\Phi(D) = \sum_{k \in \mathbb{N}} T_k$
As $T_k \in \{0, 1\}$ for each k, we have $\Phi(D) \geq 0$.

INSERT operation:
Let m be the first k where $T_k = 0$.
Insert will then set $T_0, T_1, \cdots, T_{m-1}$ from 1 to 0, and T_m from 0 to 1.
The cost of the operation is thus $1 + m$.
$\Delta\Phi(D) = m(-1) + 1$.
Thus $\hat{c} = c + \Delta\Phi(D) = 1 + m + m(-1) + 1 = 2$.

As $\hat{c}$ is constant, INSERT is amortised $O(1)$.

Accounting Method:
Invariant: Store $\$1$ on each tree in the Binomial Heap. (i.e. each T_k where $T_k = 1$.

INSERT cost: $\$2$.
We use $\$1$ for the constant time operation, and $\$1$ to store on the largest tree created.

Proof that invariant holds:
For an empty tree, the invariant holds as there is no tree.

Suppose that the invariant holds before an operation.
INSERT:
Let m be the first k where $T_k = 0$.
INSERT will do m merges to merge the trees of order 0 to $m-1$, and stop as it creates a tree of order m. The merging cost is $\$m$, which is paid for by the trees of order 0 to $m-1$.
And as $\$1$ is attached to the tree of order m, the invariant is maintained.

The invariant shows that the cost is never negative, as there cannot be a negative number of trees. As each operation costs at most $\$2$, it is amortised $O(1)$.

2. **Arbitrage**

Arbitrage takes advantage of the discrepencies in currency exchange rate to make a profit through a sequence of trades. For example, if

- 1 Singapore dollar buys 0.97 Australian dollar
- 1 Australian dollar buys 0.76 US dollars
- 1 US dollar buys 1.42 Singapore dollars,

then a smart trader who uses 1 Singapore dollar to buy Australian dollar, then converting it to US dollar and finally converting it back to Singapore dollar would get back around 1.047 dollars, earning 4.7% profit. Assume that the exchange rate is constant for a time period that is long enough to make k trades. Let $R[i, j]$ be the exchange rate table which denotes how much of currency j can be purchased using one dollar of currency i. Give a dynamic programming algorithm for finding the maximum amount that can be made by starting with one dollar of currency 1 and doing k trades, converting back to currency 1 on the k-th trade. (As usual, when describing an algorithm, you should give some indication of correctness and analysis of the running time.)

Solution: Assume that an optimal sequence of making k trades and ending up with currency c_k is $R[1, c_1]R[c_1, c_2] \cdots R[c_{k-2}, c_{k-1}]R[c_{k-1}, c_k]$. Then $R[1, c_1]R[c_1, c_2] \cdots R[c_{k-2}, c_{k-1}]$ must be an optimal sequence of making $k-1$ trades and ending up with currency c_{k-1}. If not, we can replace the sequence with a better sequence, contradicting the optimality of the the orignal sequence of k trades.

Let $M(i, k)$ denote the maximum amount of currency i that is possible by doing k trades starting from currency 1. Assume that the currencies are $1, \ldots, n$. The optimal substructure gives a recurrence assuming that the correct currency at time $k-1$ is known. As we do not know that, we maximize over all choice of currencies at time $k-1$, giving

$$M(i, k) = \max_{j \in 1, \ldots, n} \{M(j, k-1)R[j, i]\},$$

with $M(i, 1) = R[1, i]$.

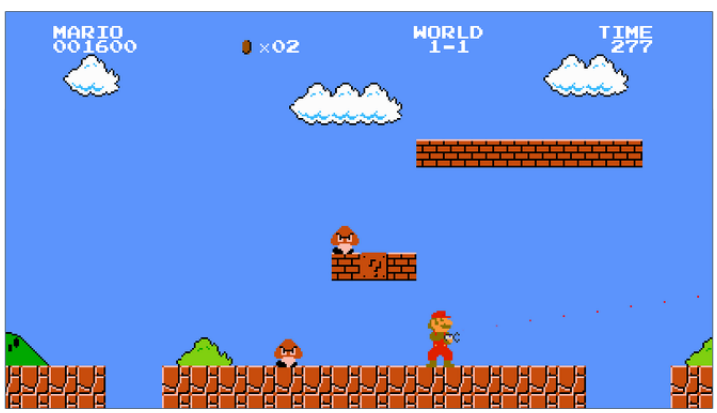
This leads to the following pseudocode:

```
for i = 1 to n
    M[i, 1] = R[1, i]
for ℓ = 2 to k
    for i = 1 to n
        M[i, ℓ] = 0
        for j = 1 to n
            if M[j, ℓ − 1]R[j, i] > M[i, ℓ]
                M[i, ℓ] = M[j, ℓ − 1]R[j, i]
```

The solution can be found in $M[1, k]$. There are three nested loops of $k - 1$, n, and n iterations respectively. Hence the runtime is $O(n^2 k)$.

3. **Would you like to build some bridges?**

   Mario is on a mission to save Princess Peach! As in the classical *Mario* games, our protagonist has to navigate through the world by jumping from one platform to another until he reaches Bowser's castle, where Princess Peach is held captive. However, Mario realised that jumping across platforms is a dangerous affair: He will perish if he were to drop in between platforms! As such, he has decided to build bridges in between platforms so that he can reach his destination safely.



   However, Mario might not have enough resource to build bridges in between all platforms. In addition, Mario is not very smart, and he cannot figure out if he has enough bridges to cover all gaps.

   To be specific, Mario's world is a one-dimensional world with n gaps which must be covered by bridges. At both ends of each gap, there are bridge stands where a bridge can be placed on. The gaps and corresponding bridge stands are of different lengths for each gap. Gap i is specified by two numbers (g_i, t_i) where g_i is the gap length and t_i is the total length of the gap plus the stands on both sides. Mario is given n bridges whereby the j-th bridge has length a_j. Bridge j can be used to cover gap i **if and only if** $g_i \leq a_j \leq t_i$.

(a) Suggest a greedy algorithm that can be used to solve Mario's problem.

Hint: One way is to sort the bridges first, then find a gap that is appropriate for the bridges from the shortest to the longest bridge.

Solution:

We say that gap i *suits* bridge j if and only if $g_i \leq a_j \leq t_i$. We will iterate from the shortest bridge to the longest bridge. For each bridge j, consider the set of remaining gaps that are suited for that bridge, i.e. remaining gaps i such that $g_i \leq a_j \leq t_i$. Choose the gap with the smallest total length within this set and assign the gap to the bridge. If all points can be assigned in this manner, then a solution exists, otherwise no solution exists.

Comment. Alternative solutions are possible. It is possible to sort the bridges in descending order. Then for each bridge, in sorted order, greedily select the gap with the largest gap length that is smaller than the bridge. It is also possible to sort the gaps in ascending order according to the total length. Then, for each gap, in sorted order, we greedily select the smallest bridge that is larger than the gap length of the gap. It is also possible to sort the gaps in descending order according to the gap lengths. In that case, for each gap, in sorted order, we greedily select the longest bridge that is smaller than the total length of the gap. In each case, similar cut and paste arguments can be made for the greedy choice.

(b) Prove the correctness of your algorithm.

Hint: First, show that the algorithm does not return a false positive (this should be straightforward). The key is to show that the algorithm does not return a false negative. One way to do that is to first argue that optimal substructure exists which allows us to to select one bridge-gap pair then optimally solve for the remaining subproblem. Then argue that greedy choice holds for the problem, i.e. there exists a solution which selects the same first bridge-gap pair as your algorithm.

Solution: Firstly, note that the above algorithm does not return a false positive as all the gaps that it assigns at every step has to suit the assigned bridge. Hence, we are concerned if the greedy algorithm returns a false negative (i.e. the greedy solution does not return a valid assignment when there is one). We will show that the algorithm will always return a valid assignment if there is one.

Consider a valid assignment of bridges to gaps. If we remove one bridge-gap pair, we need to do a valid assignment of the remaining bridge-gap pairs to obtain the valid assignment for all bridges and gaps. Hence, if we can find a bridge-gap pair that belongs to a valid assignment, we can solve the problem by solving the assigment for the remaining bridges and gaps.

We now show that our greedy choice gives us a bridge-gap pair that belongs to some valid assignment, and hence can be recursively used to solve the remaining sub-problem.

Consider a valid assignment. If our greedy first assignment matches the assignment of the valid assignment, then we are done. If not, suppose the greedy algorithm matches (g, t) to a_1 but the valid assignment assigns (g', t') to a_1 and (g, t) to a_k where $a_1 < a_k$.

We argue that we can swap the assignment of (g, t) to a_1 and (g', t') to a_k within the valid solution to get another valid solution that agrees with the greedy algorithm.

We already know that (g, t) is suited for a_1, so it remains to be argued that (g', t') is suited for a_k. To see this, note that:

- $a_1 \leq a_k$ and the valid assignment assigns (g', t') to a_1, so $g' \leq a_1 \leq a_k$.
- The greedy algorithm selects $t \leq t'$. In the valid solution, (g, t) is assigned to a_k, so $a_k \leq t$. Hence $a_k \leq t'$

As $g' \leq a_k \leq t'$, (g', t') is suited for a_k.

Comment. Other ways to structure the proof is possible, but most likely it will involve an exchange argument similar to that used above.

(c) Choose appropriate data structures to implement the above algorithm and suggests the runtime of your solution. You are encouraged to describe the steps in the execution of your algorithm using pseudocode or other means.

Solution. We will use a priority queue to keep track of the gaps that are suited for the current bridge. The priority queue will be ordered in a way such that the gap with the smallest total length will have the highest priority (such that we can identify which gap to assign to the current bridge easily). Every time a new bridge is iterated, we will push gaps with gap lengths shorter than or equal to the length of the current bridge into the priority queue. In order to quickly identify which gaps to insert, we will sort the gap by their gap length in ascending order. Upon sorting, we can simply add all intervals with gap length smaller than or equals to the current bridge into the priority queue.

Each gap will be inserted and popped once from the priority queue using $O(n \log n)$ time. Note that we also need to sort the gaps and the bridges in $O(n \log n)$ time. The total runtime is $O(n \log n)$.

Pseudocode:

```
Sort bridge lengths
Sort gaps using gap length
priority_queue<gap> pq // priority based on total length
int ptr = 0 // to keep track of which gap to push
for i = 0 to n-1
  // add all gaps where g <= bridges[i]
  while gap[ptr].g <= bridges[i] && ptr < n
    pq.push(gap[ptr])
    ptr = ptr + 1
  if(pq.empty()) return false;
  currGap = pq.top()
  if(bridges[i] > currGap.t) return false;
  else
    A[i] = currGap
    pq.pop();
```

```
return true;
```

Comment. A $\Theta(n^2)$ algorithm would be suboptimal. Other $O(n \lg n)$ solutions are possible, e.g. by augmenting balanced binary search trees.